

IFDS - Image Forgery Detection System

IFDS, görüntü manipölasyonu tespiti için geliştirilen Streamlit tabanlı bir analiz uygulamasıdır. Sistem; klasik OpenCV tabanlı özellik eşleştirme algoritmalarını ve isteğe bağlı derin öğrenme modellerini birlikte çalıştırarak yüklenen görsel için karşılaştırmalı sonuç, nihai karar ve indirilebilir rapor üretir.

Özellikler

- SIFT, SURF, AKAZE ve ORB ile klasik kopyala-yapıştır / sahtecilik sinyali analizi
- Xception CNN ve EfficientNet CNN ile isteğe bağlı AI tabanlı sınıflandırma
- Xception modeli için Grad-CAM ısı haritası desteği
- Ağırlıklı karar mekanizması ile Authentic, Tampered veya Review needed sonucu
- Görsel metadatası, algoritma karşılaştırması ve PDF/HTML rapor çıktısı
- GIF, JPG/JPEG, PNG, BMP ve TIFF format desteği

Proje Yapısı

```
.
├── app.py                # Streamlit uygulama giriş noktası
├── config/              # Uygulama ayarları ve model yolları
├── data/
│   ├── raw/            # Yerel ham veri setleri (git'e eklenmez)
│   ├── processed/      # İşlenmiş veri çıktıları (git'e eklenmez)
│   └── models/         # Yerel model ağırlıkları (git'e eklenmez)
├── docs/               # Dokümantasyon ve model metrikleri
├── notebooks/          # Eğitim notebook'ları
├── src/
│   ├── ai_models/      # Xception, EfficientNet ve Grad-CAM bileşenleri
│   ├── classical/      # SIFT, SURF, AKAZE, ORB dedektörleri
│   ├── preprocessing/  # Görüntü yükleme ve ön işleme
│   ├── reporting/      # PDF/HTML rapor üretimi
│   └── verdict.py      # Nihai karar hesaplama servisi
├── tests/              # Pytest testleri
└── ui/                 # Streamlit arayüz bileşenleri
```

Kurulum

Python 3.10 veya üzeri bir sürüm önerilir.

```
python -m venv .venv
.venv\Scripts\activate
pip install -r requirements.txt
```

macOS/Linux kullanıyorsanız sanal ortam aktivasyonu:

```
source .venv/bin/activate
```

Uygulamayı Çalıştırma

```
streamlit run app.py
```

Uygulama açıldıktan sonra bir görüntü yükleyip çalıştırılacak algoritmaları sol menüden seçebilirsiniz.

Model Dosyaları

AI modelleri opsiyoneldir. Model ağırlıkları yoksa uygulama klasik algoritmalar ve raporlama akışıyla çalışmaya devam eder.

Bu repoda ana model dosyaları Git LFS ile takip edilir:

```
data/models/xception_finetuned.h5
data/models/efficientnet_finetuned.h5
```

Repoyu klonladıktan sonra model dosyaları eksik görünürse Git LFS'i kurup aşağıdaki komutu çalıştırın:

```
git lfs pull
```

Ek model ağırlıkları, veri setleri ve eğitim çıktıları büyük dosyalar olduğu için .gitignore kapsamındadır. Eğitim metrikleri için docs/model_metrics.md ve data/models/training_results.json dosyalarına bakabilirsiniz.

Test

Geliştirme bağımlılıklarını kurup testleri çalıştırmak için:

```
pip install -r requirements-dev.txt
pytest tests -v --cov=src --cov-report=xml
```

Teslim Dokümanları

Proje ödevi kapsamında hazırlanan dokümanlar docs/ klasöründedir:

- docs/Kullanici_El_Kitapcigi_IFDS.docx: Yazılımı anlatan kullanıcı el kitapçığı
- docs/FSM_Emek_Hesabi_IFDS.docx: Use Case Point yöntemiyle FSM emek hesabı

Kalite ve Analiz Çıktıları

Kod kalitesi ve teknik dokümantasyon için üretilen çıktı dosyaları:

- docs/doxygen/html/index.html: Doxygen HTML dokümantasyonu
- docs/quality_outputs/doxygen/Doxygen_IFDS_Documentation.pdf: Doxygen HTML çıktısından oluşturulan PDF
- docs/quality_outputs/graphviz/ifds_architecture.svg: Graphviz mimari grafiği
- docs/quality_outputs/graphviz/ifds_architecture.png: Graphviz mimari grafiği görsel çıktısı
- docs/quality_outputs/graphviz/ifds_architecture.pdf: Graphviz mimari grafiği PDF çıktısı
- docs/quality_outputs/graphviz/doxygen_representative_call_graph.svg: Doxygen/Graphviz çağrı grafiği
- docs/quality_outputs/sonarqube/coverage.xml: SonarQube analizine girecek coverage çıktısı
- docs/quality_outputs/sonarqube/SONARQUBE_ANALYSIS_STATUS.md: SonarQube dashboard üretim durumu ve test/coverage özeti

CI ve SonarQube

GitHub Actions workflow'u testleri her push ve pull request için çalıştırır. SonarQube taraması için repository secrets altında şu değerler tanımlı olmalıdır:

```
SONAR_TOKEN
SONAR_HOST_URL
```

Bu secrets tanımlı değilse SonarQube adımı atlanır; test adımı çalışmaya devam eder.

Eğitim Notları

notebooks/ klasöründe Kaggle/CASIA veri setiyle model eğitimi için hazırlanmış notebook'lar bulunur. Yerel veri setlerinizi data/raw/ altında tutabilirsiniz; bu klasör GitHub'a gönderilmez.

GitHub'a Göndermeden Önce

- .env, Streamlit secrets, sanal ortamlar, veri setleri ve model ağırlıkları repoya eklenmez.
- data/raw/, data/processed/ ve data/models/ klasörleri .gitkeep ile korunur.
- Ana .h5 model dosyaları normal Git yerine Git LFS üzerinden paylaşılır.

Lisans

Bu proje MIT lisansı ile yayınlanır. Ayrıntılar için LICENSE dosyasına bakın.